

Supplemental Material for FlowBook: Dynamic Enforcement of Output Consistency in Computational Notebooks

ANONYMOUS AUTHOR(S)

This supplemental material presents the full formal development underlying FlowBook’s analysis. Section 1 formalizes an idealized model of computational notebooks, defines the notebook operations, and gives the reference top-to-bottom execution that grounds the notion of an output-consistent state (the formal counterpart of an output-consistent notebook in the main paper). Section 2 presents the instrumented semantics: the read, write, and status bookkeeping; the rerun consistency predicates; the forward and backward staleness predicates; the well-formedness invariant; and the statements of the preservation, output-consistency, and progress theorems with proof sketches. Sections 3–5 give the full proofs of those three theorems. Section 6 describes our mechanization of these proofs in Lean 4.

1 A Semantics for Interactive Computational Notebook Execution

This section formalizes an idealized model of computational notebooks and defines output consistency.

1.1 Notebook Model

A *notebook* consists of a sequence of cells, each containing source code $c \in \text{Code}$ and outputs $o \in \text{Output}$. We leave the exact form of the cell source code and output unspecified, as it is not relevant to the semantics. In addition to those components, which are visible to the user, the notebook state also includes a hidden execution store used during cell evaluation. Thus, a notebook state is a tuple $S = (C, O, \Sigma)$, where:

- $C = C_1, \dots, C_n$ is the sequence of cell source code,
- $O = O_1, \dots, O_n$ is the sequence of cell outputs, and
- $\Sigma : \text{Loc} \rightarrow \text{Val}$ is the current store.

We write C_i for the source code of cell i , O_i for its most recent output, and $\Sigma(\ell)$ for the value of location ℓ in store Σ . An output $O_i = \perp$ indicates cell i has not yet been executed.

Locations. We model state at two location granularities: top-level variables and individual columns for DataFrames. A *location* $\ell \in \text{Loc}$ identifies a unit of state that cells may read or write:

$$\ell \in \text{Loc} ::= x \mid d.c$$

where $x \in \text{Var}$ denotes a top-level variable binding, $d \in \text{Address}$ denotes a DataFrame address, and $c \in \text{String}$ denotes column c of the DataFrame at address d .

This location granularity reflects a key observation about notebook workflows: Cells communicate through global variables, and DataFrame columns are the natural unit of independent modification. When reasoning about inter-cell dependencies, this formulation enables our analysis to identify when two cells read or write to disjoint columns of the DataFrame at address d . In contrast, our analysis treats all other structures, such as lists, dictionaries, and arrays, as atomic values stored at a single location, and all reads and updates operate on these values.

We assume DataFrame addresses are immutable: address d always refers to the same DataFrame object. An operation like `df['price'] = 100` reads the DataFrame address d from the global variable `df`, and writes to the column `price` of the DataFrame at address d .

Cell Evaluation. We define cell evaluation using a big-step judgment that captures the effect of executing a single cell. The judgment

$$c; \Sigma \downarrow o \cdot \Sigma'$$

asserts that executing the code c in store Σ produces output o and resulting store Σ' .

We treat cell evaluation as a black box that models the underlying language runtime (Python, in our implementation) while abstracting over the details of expression evaluation, statement execution, library calls, and calls into native code (e.g., C extensions). Because a cell may invoke non-deterministic operations, this judgment is a *relation* rather than a function.

1.2 Notebook Operations: Syntax and Semantics

Users interact with notebooks through operations that run cells, edit their contents, or modify notebook structure:

$$op ::= \text{RUN}(i) \mid \text{EDIT}(i, c) \mid \text{INSERT}(i, c) \mid \text{DELETE}(i) \mid \text{MOVE}(i, j)$$

where:

- $\text{RUN}(i)$ executes cell i , updating its output and the store.
- $\text{EDIT}(i, c)$ replaces the source code of cell i with c .
- $\text{INSERT}(i, c)$ inserts a new cell with code c at position i , shifting cells i through n to positions $i+1$ through $n+1$.
- $\text{DELETE}(i)$ removes the cell at position i , shifting cells $i+1$ through n to positions i through $n-1$.
- $\text{MOVE}(i, j)$ relocates the cell at position i to position j .

Figure 8 specifies how each operation transforms notebook state. In these rules and below, we write $X_{i..j}$ to denote the subsequence $X_i, \dots, X_j$, and $X[i := v]$ to denote the sequence X with position i updated to v . We use $\emptyset$ for the empty store.

The key observation is that [STD-RUN] permits executing any cell at any time, regardless of whether cells above it have executed. This permissiveness models interactive notebook use, where users routinely execute cells out of order. The remaining rules are straightforward. [STD-EDIT] updates the source code without affecting the store. [STD-INSERT] and [STD-DELETE] add or remove cells, shifting indices accordingly. [STD-MOVE] relocates a cell by composing delete and insert.

1.3 Output Consistency

The flexibility of the standard semantics is the source of output-consistency problems. Because users may execute cells in any order, the current notebook state may diverge from what a clean top-to-bottom execution would produce.

We formalize the reference behavior using a top-to-bottom execution judgment. The judgment

$$\frac{[\text{STD-TOP-TO-BOTTOM-EXECUTION}] \quad C_1; \emptyset \downarrow O_1 \cdot \Sigma_1 \quad C_2; \Sigma_1 \downarrow O_2 \cdot \Sigma_2 \quad \dots \quad C_n; \Sigma_{n-1} \downarrow O_n \cdot \Sigma_n}{C \downarrow O \cdot \Sigma_n}$$

asserts that executing cells $C_1, \dots, C_n$ top-to-bottom from the empty store produces outputs $O_1, \dots, O_n$ and final store Σ via the intermediate stores $\Sigma_1, \dots, \Sigma_{n-1}$.

Definition 1.1 (Output-Consistent State). A notebook state $S = (C, O, \Sigma)$ is *output consistent* if there exists Σ' such that $C \downarrow O \cdot \Sigma'$.

That is, a notebook state is output consistent if and only if some top-to-bottom execution of all cells from an empty store produces exactly the outputs currently recorded in the notebook. The

$$\begin{array}{c}
 \boxed{S \xrightarrow{op} S'} \\
 \\
 \text{[STD-RUN]} \\
 \frac{C_i; \Sigma \downarrow o \cdot \Sigma' \quad O' = O[i := o]}{(C, O, \Sigma) \xrightarrow{\text{RUN}(i)} (C, O', \Sigma')} \\
 \\
 \text{[STD-INSERT]} \\
 \frac{C' = C[i := c] \quad O' = O[i := \perp]}{(C, O, \Sigma) \xrightarrow{\text{INSERT}(i, c)} (C', O', \Sigma)} \\
 \\
 \text{[STD-MOVE-DOWN]} \\
 \frac{\begin{array}{c} s < d \\ S \xrightarrow{\text{DELETE}(s)} S'' \xrightarrow{\text{INSERT}(d-1, C_s)} S' \end{array}}{S \xrightarrow{\text{MOVE}(s, d)} S'} \\
 \\
 \text{[STD-EDIT]} \\
 \frac{C' = C[i := c]}{(C, O, \Sigma) \xrightarrow{\text{EDIT}(i, c)} (C', O, \Sigma)} \\
 \\
 \text{[STD-DELETE]} \\
 \frac{\begin{array}{c} C' = C_{1..i-1}, C_{i+1..n} \\ O' = O_{1..i-1}, O_{i+1..n} \end{array}}{(C, O, \Sigma) \xrightarrow{\text{DELETE}(i)} (C', O', \Sigma)} \\
 \\
 \text{[STD-MOVE-UP]} \\
 \frac{\begin{array}{c} s > d \\ S \xrightarrow{\text{DELETE}(s)} S'' \xrightarrow{\text{INSERT}(d, C_s)} S' \end{array}}{S \xrightarrow{\text{MOVE}(s, d)} S'}
 \end{array}$$

Fig. 8. The standard semantics operates on notebook state $S = (C, O, \Sigma)$ where C is the sequence of cell source code, O is the sequence of cell outputs, and Σ is the variable store.

final stores Σ and Σ' need not be identical. Σ reflects the actual interactive execution history while Σ' reflects the reference execution, but the outputs O must agree.

2 Dynamic Analysis for Output Consistency

This section presents FlowBook's dynamic analysis for output consistency as an instrumented semantics. It ensures output consistency by tracking read and write sets of locations for each executed cell, enforcing rerun consistency, and maintaining cell status (either STALE or CLEAN). We prove that this instrumentation preserves a well-formedness invariant and that well-formed states with all clean cells are output consistent.

2.1 Instrumentation State

Figure 9 presents the instrumented semantics for an instrumented state $S \cdot I$ that augments the standard state S with *instrumentation* $I = (T, R, W)$ where:

- $T_i \in \{\text{CLEAN}, \text{STALE}\}$ is a *tag* that records the status of cell i ,
- $R_i \subseteq \text{Loc}$ is the set of locations read by cell i , and
- $W_i \subseteq \text{Loc}$ is the set of locations written by cell i .

The status $T_i = \text{CLEAN}$ indicates that cell i 's recorded output remains valid with respect to the current store. The status $T_i = \text{STALE}$ indicates that the cell must be re-executed. Initially, all cells are stale with empty read and write sets.

2.2 Instrumented Cell Evaluation

Our instrumented semantics is based on an instrumented cell evaluation judgment

$$c; \Sigma \Downarrow o \cdot \Sigma' \cdot r \cdot w$$

that extends standard cell evaluation $c; \Sigma \downarrow o \cdot \Sigma'$ to also record the set of read locations $r \subseteq \text{Loc}$ in Σ that are read during evaluation of c , and the set of write locations $w \subseteq \text{Loc}$ where Σ' is updated

$$\begin{array}{c}
\boxed{S \cdot I \xRightarrow{op} S' \cdot I'} \\
\text{[INST-RUN]} \\
\begin{array}{l}
S = (C, O, \Sigma) \quad C_i; \Sigma \Downarrow o \cdot \Sigma' \cdot r \cdot w \\
S' = (C, O[i := o], \Sigma') \\
R' = R[i := r] \quad W' = W[i := w] \\
R' \text{ and } W' \text{ are rerun consistent for } i \\
T'_j = \begin{cases} \text{CLEAN} & \text{if } j = i \\ \text{STALE} & \text{if FORWARDSTALE}(R, W, W', i, j) \\ \text{STALE} & \text{if BACKWARDSTALE}(W, W', i, j) \\ T_j & \text{otherwise} \end{cases} \\
\hline
S \cdot (T, R, W) \xRightarrow{\text{RUN}(i)} S' \cdot (T', R', W')
\end{array} \\
\text{[INST-INSERT]} \\
\begin{array}{l}
S \xrightarrow{\text{INSERT}(i, c)} S' \\
R' = R[i := \emptyset] \quad W'' = W[i := \emptyset] \quad T' = T[i := \text{STALE}] \\
\hline
S \cdot (T, R, W) \xRightarrow{\text{INSERT}(i, c)} S' \cdot (T', R', W')
\end{array} \\
\text{[INST-MOVE-DOWN]} \\
\begin{array}{l}
s < d \\
S \cdot I \xrightarrow{\text{DELETE}(s)} S'' \cdot I'' \xrightarrow{\text{INSERT}(d-1, C_s)} S' \cdot I' \\
\hline
S \cdot I \xRightarrow{\text{MOVE}(s, d)} S' \cdot I'
\end{array} \\
\text{[INST-EDIT]} \\
\begin{array}{l}
S \xrightarrow{\text{EDIT}(i, c)} S' \quad T' = T[i := \text{STALE}] \\
\hline
S \cdot (T, R, W) \xRightarrow{\text{EDIT}(i, c)} S' \cdot (T', R, W)
\end{array} \\
\text{[INST-DELETE]} \\
\begin{array}{l}
S \xrightarrow{\text{DELETE}(i)} S' \\
R'' = R[i := \emptyset] \quad W'' = W[i := \emptyset] \\
T'_j = \begin{cases} \text{STALE} & \text{if FORWARDSTALE}(R, W, W'', i, j) \\ \text{STALE} & \text{if BACKWARDSTALE}(W, W'', i, j) \\ T_j & \text{otherwise} \end{cases} \\
R' = R''_{1..i-1}, R''_{i+1..n} \quad W' = W''_{1..i-1}, W''_{i+1..n} \\
T' = T'_{1..i-1}, T'_{i+1..n} \\
\hline
S \cdot (T, R, W) \xRightarrow{\text{DELETE}(i)} S' \cdot (T', R', W')
\end{array} \\
\text{[INST-MOVE-UP]} \\
\begin{array}{l}
s > d \\
S \cdot I \xrightarrow{\text{DELETE}(s)} S'' \cdot I'' \xrightarrow{\text{INSERT}(d, C_s)} S' \cdot I' \\
\hline
S \cdot I \xRightarrow{\text{MOVE}(s, d)} S' \cdot I'
\end{array}
\end{array}$$

Fig. 9. The instrumented semantics operates on $S \cdot I$. $I = (T, R, W)$ tracks cell status, read sets, and write sets.

from Σ . Note that a cell such as

$$x = \emptyset; \quad x = x + 1$$

does not technically read the initial value of x in the store, so this cell would have an empty read set $r = \emptyset$ and a write set $w = \{x\}$. The FlowBook implementation instruments the underlying run-time system to capture these sets.

The [INST-RUN] rule uses this extended judgment to evaluate cell i via $C_i; \Sigma \Downarrow o \cdot \Sigma' \cdot r \cdot w$. That rule requires that the read and write sets satisfy rerun consistency for the cell, meaning that they capture accesses are rerun consistent, which is formally defined as follows.

Definition 2.1 (Rerun Consistent Accesses). Instrumentation state R and W are *rerun consistent* for cell i if these predicates hold:

- $\text{NoReadAndWrite}(R, W, i)$: cell i does not read and write the same location.
- $\text{WriteAboveRead}(R, W, i)$: every location read by i was written by a cell above i .
- $\text{NoReadAboveWrite}(R, W, i)$: cell i does not read locations written by cells below i .
- $\text{NoWriteBelowRead}(R, W, i)$: cell i does not overwrite locations read by cells above i .

We define these predicates formally as follows; the main paper illustrates them in Figure 3.

$$\begin{array}{ll}
\text{NoReadAndWrite}(R, W, i) & \stackrel{\text{def}}{=} R_i \cap W_i = \emptyset \\
\text{WriteAboveRead}(R, W, i) & \stackrel{\text{def}}{=} R_i \subseteq (\cup W_{1..i-1}) \\
\text{NoReadAboveWrite}(R, W, i) & \stackrel{\text{def}}{=} R_i \cap (\cup W_{i+1..n}) = \emptyset \\
\text{NoWriteBelowRead}(R, W, i) & \stackrel{\text{def}}{=} W_i \cap (\cup R_{1..i-1}) = \emptyset
\end{array}$$

The [INST-RUN] rule also sets to stale any cells whose inputs may have been invalidated by the executed cell. That includes cells C_j below i whose reads R_j or writes W_j conflict with the executed cell's writes W'_i or with locations $W_i \setminus W'_i$ that used to be written by cell i but no longer are:

$$\text{FORWARDSTALE}(R, W, W', i, j) \stackrel{\text{def}}{=} j > i \wedge (W_i \cup W'_i) \cap (R_j \cup W_j) \neq \emptyset$$

If a cell i previously wrote to a location ℓ but no longer does (i.e., $\ell \in W_i \setminus W'_i$) then we also mark the nearest writer of ℓ above cell i as stale. This ensures that the write is re-executed to produce a store consistent with top-to-bottom execution.

$$\text{LASTWRITER}(W, i, \ell) \stackrel{\text{def}}{=} \max\{k < i \mid \ell \in W_k\}$$

$$\text{BACKWARDSTALE}(W, W', i, j) \stackrel{\text{def}}{=} j < i \wedge j = \text{LASTWRITER}(W, i, \ell) \text{ for some } \ell \in W_i \setminus W'_i$$

The [INST-DELETE] rule uses the same logic to identify stale cells after deleting cell i : these are cells whose reads or writes conflict with what the deleted cell had written. The main paper illustrates these forward and backward staleness conditions in Figure 4 on a collection of small example notebooks (see also Section 3.5 of the main paper). The remaining rules are straightforward.

2.3 Well-Formedness

The instrumented semantics maintains a *well-formedness invariant* that characterizes when a notebook state permits output-consistent execution.

Definition 2.2 (Well-Formed State). An instrumented state $S \cdot I = (C, O, \Sigma) \cdot (T, R, W)$ is *well-formed* if for every i with $T_i = \text{CLEAN}$, there exists Σ' such that:

- (1) $C_i; \Sigma \Downarrow O_i \cdot \Sigma' \cdot R_i \cdot W_i$,
- (2) Σ and Σ' agree except on $\bigcup W_{i+1..n}$, the set of locations written by cells below i .
- (3) R and W are rerun consistent for i .

That is, every clean cell i can be re-executed from the current store to produce its recorded output O_i and read and write sets R_i and W_i , and only changes locations in Σ' that are overwritten by writes below i .

2.4 Correctness

We establish correctness by first showing that every notebook operation preserves well-formedness. The three theorems below make precise the preservation, output consistency, and progress guarantees stated in Section 3.6; their full proofs appear in Sections 3–5.

THEOREM 2.3 (PRESERVATION). *If $S \cdot I$ is well-formed and $S \cdot I \xRightarrow{op} S' \cdot I'$, then $S' \cdot I'$ is well-formed.*

PROOF SKETCH. The proof proceeds by cases on the operation. For [INST-RUN], the key insight is that the rule antecedents ensure the executed cell satisfies rerun consistency, while the staleness predicates mark any cell that would violate rerun consistency. For [INST-EDIT], the edited cell becomes stale, preserving rerun consistency vacuously. For structural operations, cells that would violate rerun consistency after index shifting are marked stale. $\square$

Second, well-formed states with all cells clean are output consistent:

THEOREM 2.4 (OUTPUT CONSISTENCY). *If $S \cdot I = (C, O, \Sigma) \cdot (T, R, W)$ is well-formed and $T_i = \text{CLEAN}$ for all i , then (C, O, Σ) is output consistent, i.e. there exists Σ' such that $C \Downarrow O \cdot \Sigma'$.*

PROOF SKETCH. By induction on i . The inductive hypothesis states that executing $C_1, \dots, C_i$ top-to-bottom from the empty store produces outputs $O_1, \dots, O_i$ and a store that agrees with Σ on

locations in $W_{1..i}$ that do not conflict with writes below i ($W_{i+1..n}$). The base case is trivial. For the inductive step, rerun consistency of cell i together with WRITEABOVEREAD and NOREADABOVEWRITE ensure that the last execution of cell i read the same values as it would in a top-to-bottom execution, thereby producing the same output O_i . $\square$

Preservation and output consistency establish that well-formedness is maintained and that all-clean states are output consistent, but not that an all-clean state is *reachable*. The natural strategy for reaching one repeatedly runs the first stale cell. For non-deterministic cells this strategy need not terminate: two cells with empty read sets that alternately write $\{a\}$ and $\{b\}$ mark each other stale forever. We therefore assume that cells are deterministic, in a sequential sense. Evaluating code c from store Σ reads locations one at a time; let $\rho(c, \Sigma)$ be the sequence of locations it reads, up to completion or failure.

ASSUMPTION 2.5 (DETERMINISM). *If $c; \Sigma \Downarrow o \cdot \Sigma' \cdot r \cdot w$, then r is the set of locations in $\rho(c, \Sigma)$. Moreover, for every store $\hat{\Sigma}$:*

- (1) *if $\hat{\Sigma}$ agrees with Σ on the first k locations of $\rho(c, \Sigma)$, then $\rho(c, \hat{\Sigma})$ begins with the first $k + 1$ locations of $\rho(c, \Sigma)$, or equals $\rho(c, \Sigma)$ if $k = |\rho(c, \Sigma)|$; and*
- (2) *if $\hat{\Sigma}$ agrees with Σ on r and $c; \hat{\Sigma} \Downarrow \hat{o} \cdot \hat{\Sigma}' \cdot \hat{r} \cdot \hat{w}$, then $\hat{w} = w$ and $\hat{\Sigma}'$ agrees with Σ' on w .*

That is, the location a cell reads next, and the locations and values it writes, depend only on the values it has read so far; its output may still differ. Python executes a cell's statements in order, so every deterministic cell satisfies Assumption 2.5.

The order of reads matters. Requiring only that a cell's read set depend on the values it reads does not ensure termination. Consider three cells that write $a := 0$, $b := 0$, and $c := 0$, followed by a cell U that reads $\{a, b\}$ and writes $c := 1$ if $a \neq 0 = b$; reads $\{b, c\}$ and writes $a := 1$ if $b \neq 0 = c$; reads $\{c, a\}$ and writes $b := 1$ if $c \neq 0 = a$; and reads all three otherwise. From the well-formed state in which $a = 1$, $b = c = 0$, and only U is stale, having last written a , the strategy runs U , then the writer of a , U , the writer of c , U , and the writer of b , and returns to that state. Each run of U stops writing the location it wrote last, so BACKWARDSTALE marks that location's writer, whose rerun marks U again. No cell that reads one location at a time behaves like U : the location it reads first would belong to every read set.

THEOREM 2.6 (PROGRESS). *Under Assumption 2.5, if $S \cdot I$ is well-formed, then every sequence of [INST-RUN] steps from $S \cdot I$, each of which runs the first stale cell, is finite. When no such step exists, the current state $S' \cdot I'$ is well-formed and either:*

- (1) *all cells are clean, so S' is output consistent by Theorem 2.4; or*
- (2) *the first stale cell i is stuck: there is no $S'' \cdot I''$ such that $S' \cdot I' \xrightarrow{\text{Run}(i)} S'' \cdot I''$, because the rerun consistency checks of [INST-RUN] fail or the cell raises an exception.*

PROOF SKETCH. Every state reached is well-formed by Theorem 2.3, and a state whose first stale cell has no run is all-clean or stuck by definition, so the content of the theorem is termination. Let E be the top-to-bottom execution of the current code, which Assumption 2.5 makes unique up to outputs. Well-formedness implies that every cell above the first stale cell f records E 's read and write sets, and that the store agrees with E on the locations those cells write and no cell at or below f writes. Hence a run of f either behaves as in E or first diverges from E at a location p that f itself wrote. In the latter case, NOREADANDWRITE removes p from W_f , and BACKWARDSTALE marks the cell above f whose rerun restores p . Runs of cells above f behave as in E and keep their write sets, so they move the first stale cell down, and each later run of f agrees with E on a strictly longer prefix of E 's read sequence for f . A lexicographic measure on these two quantities decreases with every run until the first stale cell moves below f . $\square$

Together, these three theorems establish FlowBook's correctness: starting from an initially well-formed state (all cells stale), every sequence of operations maintains well-formedness (Theorem 2.3); for deterministic cells, repeatedly running the first stale cell terminates (Theorem 2.6); and when all cells are clean, the notebook is output consistent (Theorem 2.4). The full proofs appear in Sections 3–5 below.

3 Preservation: Notebook Operations Preserve Well-Formedness

In this section, we use $W_{i..j}$ to abbreviate $W_i \cup \dots \cup W_j$.

THEOREM 2.3 (PRESERVATION). *If $S \cdot I$ is well-formed and $S \cdot I \xRightarrow{op} S' \cdot I'$, then $S' \cdot I'$ is well-formed.*

PROOF. The proof proceeds by cases on the operation.

- [INST-EDIT]. $\text{EDIT}(i, c)$ sets $T'_i = \text{STALE}$ and leaves R and W unchanged. Every cell that was CLEAN in I remains CLEAN in I' with the same witness.
- [INST-RUN]. Suppose $S \cdot I = (C, O, \Sigma) \cdot (T, R, W)$ is well-formed and

$$S \cdot I \xRightarrow{\text{RUN}(i)} S' \cdot I' = (C, O', \Sigma') \cdot (T', R', W')$$

where R' and W' agree with R and W except at position i . We show $S' \cdot I'$ is well-formed. Let j be any index with $T'_j = \text{CLEAN}$. We exhibit Σ'' such that

$$C_j; \Sigma' \Downarrow O'_j \cdot \Sigma'' \cdot R'_j \cdot W'_j$$

with Σ' and Σ'' agreeing except on $W_{j+1..n}$ and where R' and W' are rerun consistent for j . The proof splits into three subcases.

- SUBCASE $j < i$. $\text{RUN}(i)$ sets $T'_i = \text{CLEAN}$ and may mark cells STALE via FORWARDSTALE or BACKWARDSTALE. Since $T'_j = \text{CLEAN}$ and $j < i$, cell j was not marked STALE, so $T_j = \text{CLEAN}$.

Well-formedness of $S \cdot I$ yields Σ''' such that

$$C_j; \Sigma \Downarrow O_j \cdot \Sigma''' \cdot R_j \cdot W_j$$

where Σ and Σ''' agree except on $W_{j+1..n}$ and where R and W are rerun consistent for j . Since $j < i$, we have $R'_j = R_j$ and $W'_j = W_j$. $\text{WRITEABOVEWRITE}(R', W', j)$ holds because $W'_{1..j-1} = W_{1..j-1}$. $\text{NOREADABOVEWRITE}(R', W', j)$ holds: any violation would require $R_j \cap W'_i \neq \emptyset$, but then the

$$\text{NWRITEBELOWREAD}(R', W', i)$$

for $\text{RUN}(i)$ would fail.

The following diagram commutes:

$$\begin{array}{ccc} \Sigma & \xrightarrow{C_j} & \Sigma''' \\ c_i \downarrow & & \downarrow \\ \Sigma' & \xrightarrow{C_j} & \Sigma'' \end{array}$$

The stores Σ and Σ' differ only on W'_i . Since $R_j \cap W'_i = \emptyset$, running C_j from Σ' yields the same behavior as from Σ . Thus Σ' and Σ'' agree except on $W_{j+1..n}$. Hence $S' \cdot I'$ is well-formed.

- SUBCASE $j = i$. The rule INST-RUN sets $T'_i = \text{CLEAN}$. The antecedent of the rule establishes that R' and W' are rerun consistent for i .

The evaluation $C_i; \Sigma \Downarrow O'_i \cdot \Sigma' \cdot R'_i \cdot W'_i$ is given by the rule. $\text{NoReadAndWrite}(R', W', i)$ implies cell i does not read its own writes. Thus $C_i; \Sigma' \Downarrow O'_i \cdot \Sigma' \cdot R'_i \cdot W'_i$. The store Σ' agrees with itself except on $W_{i+1..n}$. Hence $\Sigma' \cdot I'$ is well-formed.

- SUBCASE $j > i$. Since $T'_j = \text{CLEAN}$ and $j > i$, cell j was not marked STALE by $\text{ForwardStale}(R', W', i, j)$. Thus $W'_i \cap (R_j \cup W_j) = \emptyset$ and $T_j = \text{CLEAN}$. Well-formedness of $S \cdot I$ yields Σ''' such that

$$C_j; \Sigma \Downarrow O_j \cdot \Sigma''' \cdot R_j \cdot W_j$$

where Σ and Σ''' agree except on $W_{j+1..n}$ and where R and W are rerun consistent for j . Since $j > i$, we have $R'_j = R_j$, $W'_j = W_j$, and $O'_j = O_j$, and rerun consistency of R' and W' at j holds. The following diagram commutes:

$$\begin{array}{ccc} \Sigma & \xrightarrow{C_j} & \Sigma''' \\ C_i \downarrow & & \downarrow \\ \Sigma' & \xrightarrow{C_j} & \Sigma'' \end{array}$$

The stores Σ and Σ' differ only on W'_i . Since $W'_i \cap R_j = \emptyset$, running C_j from Σ' yields the same result. Thus Σ' and Σ'' agree except on $W'_{j+1..n}$, and well-formedness holds.

- [INST-INSERT]. Suppose $S \cdot I \xrightarrow{\text{INSERT}(i,c)} S' \cdot I'$. The rule sets $T'_i = \text{STALE}$ with $R'_i = \emptyset$ and $W'_i = \emptyset$. All other cells shift indices: for $k \geq i$, cell k in I becomes cell $k + 1$ in I' . Let j be any index with $T'_j = \text{CLEAN}$. Then $j \neq i$ (since $T'_i = \text{STALE}$). If $j < i$, cell j is unchanged: $T'_j = T_j$, $R'_j = R_j$, $W'_j = W_j$. If $j > i$, cell j was cell $j - 1$ in I : $T'_j = T_{j-1}$, $R'_j = R_{j-1}$, $W'_j = W_{j-1}$. Since $W'_i = \emptyset$, inserting at i does not affect WriteAboveRead or NoReadAboveWrite . The set $W'_{j+1..n}$ equals $W_{j+1..n}$ (for $j < i$) or $W_{j..n}$ (for $j > i$), plus $W'_i = \emptyset$. The witness store from I serves as the witness in I' .
- [INST-DELETE]. Suppose $S \cdot I \xrightarrow{\text{DELETE}(i)} S' \cdot I'$. The rule removes cell i and marks cell j stale if $R_j \cap W_i \neq \emptyset$ for all j . For $k > i$, cell k in I becomes cell $k - 1$ in I' . Let j be any index with $T'_j = \text{CLEAN}$. Then $R_j \cap W_i = \emptyset$ (otherwise j would be STALE). If $j < i$, cell j is unchanged in R and W . If $j \geq i$, cell j was cell $j + 1$ in I . $\text{WriteAboveRead}(R', W', j)$ holds because $R_j \subseteq W_{1..j-1}$ and we removed W_i where $R_j \cap W_i = \emptyset$. Also,

$$\text{NoReadAboveWrite}(R', W', j)$$

holds. The witness store from I serves as the witness in I' : cell j did not read W_i , so removing those writes does not affect the evaluation. Well-formedness holds.

- [INST-MOVE]. $\text{MOVE}(s, d)$ is defined as $\text{DELETE}(s)$ followed by $\text{INSERT}(d', C_s)$, where $d' = d - 1$ if $s < d$ and $d' = d$ if $s > d$. Preservation follows by composing the two cases above: if $S \cdot I$ is well-formed, then $\text{DELETE}(s)$ yields a well-formed $S'' \cdot I''$, and $\text{INSERT}(d', C_s)$ yields a well-formed $S' \cdot I'$.

All cases establish well-formedness of $S' \cdot I'$. □

4 Output Consistency: Well-Formed All-Clean States Are Output Consistent

THEOREM 2.4 (OUTPUT CONSISTENCY). *If $S \cdot I = (C, O, \Sigma) \cdot (T, R, W)$ is well-formed and $T_i = \text{CLEAN}$ for all i , then there exists Σ' such that $C \Downarrow O \cdot \Sigma'$.*

PROOF. The proof is by induction on i . Define

$$P(i) \stackrel{\text{def}}{=} C_{1..i} \downarrow O_{1..i} \cdot \Sigma_i \quad \text{for some } \Sigma_i \text{ where } \Sigma \text{ and } \Sigma_i \text{ agree on } W_{1..i} \setminus W_{i+1..n}.$$

- BASE CASE. $P(0)$ holds vacuously: $\varepsilon \downarrow \varepsilon \cdot \emptyset$.
- INDUCTIVE STEP. Suppose $P(i-1)$ holds. Then $C_{1..i-1} \downarrow O_{1..i-1} \cdot \Sigma_{i-1}$ where Σ and Σ_{i-1} agree on $W_{1..i-1} \setminus W_{i..n}$. We show $C_i; \Sigma_{i-1} \downarrow O_i \cdot \Sigma_i$. Since $T_i = \text{CLEAN}$, well-formedness yields Σ' such that

$$C_i; \Sigma \Downarrow O_i \cdot \Sigma' \cdot R_i \cdot W_i, \quad \text{where } \Sigma \text{ and } \Sigma' \text{ agree except on } W_{i+1..n}.$$

We have that:

$$\text{NoReadAndWrite}(R, W, i) \implies R_i \cap W_i = \emptyset$$

$$\text{WriteAboveRead}(R, W, i) \implies R_i \subseteq W_{1..i-1}$$

$$\text{NoReadAboveWrite}(R, W, i) \implies R_i \cap W_{i..n} = \emptyset$$

and:

$$R_i \subseteq W_{1..i-1} \setminus W_{i..n}$$

By the inductive hypothesis, Σ and Σ_{i-1} agree on R_i . It follows that

$$C_i; \Sigma_{i-1} \downarrow O_i \cdot \Sigma_i$$

Thus Σ_i and Σ agree on $W_{1..i} \setminus W_{i+1..n}$, establishing $P(i)$.

□

5 Progress: Running Stale Cells Terminates

In this section, a *run* is an [INST-RUN] step of the first stale cell. Runs do not change the code C , so we can compare every state with a single top-to-bottom execution of C .

The reference execution. Let $\Sigma_1^E = \emptyset$. If Σ_k^E is defined and $C_k; \Sigma_k^E \Downarrow o \cdot \Sigma' \cdot r \cdot w$ for some o , Σ' , r , and w , then $\Sigma_{k+1}^E = \Sigma'$; otherwise Σ_{k+1}^E is undefined. By part 2 of Assumption 2.5, all such evaluations write the same values to the same locations and leave all other locations unchanged, so Σ_{k+1}^E is well defined; only their outputs may differ. Let $\rho_k = \rho(C_k, \Sigma_k^E)$ be the read sequence of cell k in this execution, and let

$$X_k \stackrel{\text{def}}{=} W_{1..k-1} \setminus W_{k..n}$$

be the locations written above cell k and not at or below it. Cell k is *settled* if Σ_k^E is defined, $C_k; \Sigma_k^E \Downarrow o \cdot \Sigma' \cdot R_k \cdot W_k$ for some o and Σ' , and $R_k \cap W_k = \emptyset$. That is, cell k records the read and write sets of the reference execution.

LEMMA 5.1 (CLEAN PREFIX). *If $S \cdot I = (C, O, \Sigma) \cdot (T, R, W)$ is well-formed and $T_k = \text{CLEAN}$ for all $k < m$, then Σ_m^E is defined, every cell $k < m$ is settled, and Σ agrees with Σ_m^E on X_m .*

PROOF. By induction on m , following the proof of Theorem 2.4. For $m = 1$, $X_1 = \emptyset$. Suppose the lemma holds for m and $T_m = \text{CLEAN}$. Rerun consistency of cell m gives $R_m \subseteq X_m$, so Σ and Σ_m^E agree on R_m , and the evaluation $C_m; \Sigma \Downarrow O_m \cdot \Sigma' \cdot R_m \cdot W_m$ that well-formedness provides is also available from Σ_m^E , with the same read set, write set, and written values. Hence Σ_{m+1}^E is defined, and cell m is settled because NoReadAndWrite gives $R_m \cap W_m = \emptyset$. Finally, $X_{m+1} = (W_m \setminus W_{m+1..n}) \cup (X_m \setminus W_m)$. On the first set, Σ_{m+1}^E holds the values that cell m writes, and so does Σ , which agrees with Σ' outside $W_{m+1..n}$. On the second, Σ_{m+1}^E agrees with Σ_m^E , and hence with Σ . □

LEMMA 5.2 (RUN ANALYSIS). Let $S \cdot I = (C, O, \Sigma) \cdot (T, R, W)$ be well-formed with first stale cell f , and let $S \cdot I \xrightarrow{RUN(f)} S' \cdot I'$, where cell f evaluates with read set r and write set w . Then Σ_f^E is defined, and either:

- (a) Σ agrees with Σ_f^E on ρ_f ; then r is the set of locations in ρ_f , and $C_f; \Sigma_f^E \Downarrow o' \cdot \Sigma'' \cdot r \cdot w$ for some o' and Σ'' ; or
- (b) $\rho_f = \pi p \pi'$, where Σ agrees with Σ_f^E on π but not at p ; then r contains p and the locations in π , and $p \in W_f \setminus w$.

PROOF. By Lemma 5.1, Σ_f^E is defined and Σ agrees with it on X_f . In case (a), part 1 of Assumption 2.5 gives $\rho(C_f, \Sigma) = \rho_f$, so r is the set of locations in ρ_f , and because Σ and Σ_f^E agree on r , the evaluation from Σ is also available from Σ_f^E . In case (b), part 1 shows that $\rho(C_f, \Sigma)$ begins with πp . The checks of [INST-RUN] then give $p \in W_{1..f-1}$ (WRITEABOVEWRITE), $p \notin W_{f+1..n}$ (NOREADABOVEWRITE), and $p \notin w$ (NOREADANDWRITE). Since Σ and Σ_f^E differ at p , $p \notin X_f$, so $p \in W_f$. $\square$

THEOREM 2.6 (PROGRESS). Under Assumption 2.5, if $S \cdot I$ is well-formed, then every sequence of runs from $S \cdot I$ is finite. When no run exists, the current state is well-formed and either all cells are clean or the first stale cell is stuck.

PROOF. Every state reached is well-formed by Theorem 2.3, and a state whose first stale cell has no run is all-clean or stuck by definition. It remains to show that every sequence of runs is finite.

We group runs into phases. A *phase for cell m* begins in a state whose first stale cell is m and lasts while the first stale cell is at or above m . By Lemma 5.1, Σ_m^E and hence ρ_m are defined. For some κ with $0 \leq \kappa \leq |\rho_m| + 1$, every state of the phase satisfies:

- (P1) every cell above m is settled;
- (P2) the first $\min(\kappa, |\rho_m|)$ locations of ρ_m lie in X_m ; and
- (P3) if $\kappa = |\rho_m| + 1$, then $C_m; \Sigma_m^E \Downarrow o \cdot \Sigma' \cdot R_m \cdot W_m$ for some o and Σ' .

Initially $\kappa = 0$, and (P1) holds by Lemma 5.1. Consider a run of the first stale cell $f \leq m$, with read set r and write set w .

Case $f < m$. Cell f is settled by (P1), so case (b) of Lemma 5.2 cannot arise: it would place p in W_f and in ρ_f , whose locations form R_f by Assumption 2.5, but $R_f \cap W_f = \emptyset$. In case (a), part 2 of Assumption 2.5, applied to the evaluation of Lemma 5.2 and the one that makes f settled, gives $w = W_f$; also $r = R_f$. The run therefore leaves R and W unchanged, so (P1)–(P3) still hold. Since $W_f \setminus w = \emptyset$, BACKWARDSTALE marks nothing, and FORWARDSTALE marks only cells below f , so the first stale cell moves below f .

Case $f = m$. All cells above m are clean, so by Lemma 5.1 and (P2), Σ agrees with Σ_m^E on the first $\min(\kappa, |\rho_m|)$ locations of ρ_m . In case (b) of Lemma 5.2, $\rho_m = \pi p \pi'$ with $|\pi| \geq \kappa$, and cell m has now read every location in πp , so the checks of [INST-RUN] place them in X_m ; (P2) and (P3) hold with $|\pi| + 1 > \kappa$ in place of κ . In case (a), if $\kappa = |\rho_m| + 1$, then (P3) and part 2 of Assumption 2.5 give $w = W_m$, so no cell above m is marked, every cell through m is clean, and the phase ends. Otherwise, (P2) and (P3) hold with $|\rho_m| + 1$ in place of κ : the checks place the locations of ρ_m , which form r , in X_m , and Lemma 5.2 gives (P3). In both cases (P1) still holds, since the run does not change the cells above m .

Every run in the phase therefore either ends the phase or decreases $(|\rho_m| + 1 - \kappa, n - f)$ lexicographically. Hence the phase contains at most $|\rho_m| + 2$ runs of cell m , each followed by at most $m - 1$ runs of cells above m . A phase ends stuck or in a state whose first stale cell, if any, lies

below m , where the next phase begins. There are thus at most n phases, for strictly increasing cells, and at most $\sum_m m (|\rho_m| + 2)$ runs in all. $\square$

6 Mechanization in Lean.

We have mechanized the formal development in Lean 4, using only Lean's core library. The mechanization covers the notebook semantics and output consistency (Section 1), the instrumented semantics and well-formedness invariant (Section 2), and complete proofs of the preservation, output-consistency, and progress theorems. It contains no sorrys and no custom axioms: every theorem depends only on Lean's three standard axioms. As in this supplement, cell evaluation is a black-box relation. The preservation and output-consistency proofs assume exactly two of its properties: evaluation does not change the store outside its write set, and evaluation depends only on the locations it reads. The progress proof additionally assumes Assumption 2.5 and follows Section 5. To show that these assumptions are satisfiable, we instantiate them for a small concrete cell language and instantiate all three theorems at that model. We also check the example of Section 2.4: the cell U satisfies the set-based form of determinism, yet running the first stale cell cycles forever. The mechanization also includes an executable reference kernel for that language. Its rerun consistency checks and staleness propagation are proved to agree with the relational definitions, and each operation it accepts is proved to be a step of the instrumented semantics. The kernel replays every scenario of Figures 3 and 4 and reports the violations and staleness tags shown there. The Lean proof is available at: <https://anonymous.4open.science/r/FlowBook/>.